

Master Thesis

Title

by

Mihir Mahesh Ambli

7th October, 2026

University of Kaiserslautern-Landau
Department of Computer Science
Distributed Systems Lab

Examiner: Prof. Dr.-Ing. Jens B. Schmitt
M. Sc. Daniel Reti

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die von mir vorgelegte Arbeit mit dem Thema „Title “ selbstständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel (auch generative KI-Systeme) vollständig angegeben habe und dass ich die Stellen der Arbeit — einschließlich Tabellen und Abbildungen —, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind unter Angabe der Quelle als Entlehnung kenntlich gemacht habe. Sofern der Einsatz generativer KI-Systeme über eine reine Grammatikprüfung und Stilverbesserung hinausgeht, ist der Einsatz nachvollziehbar kenntlich gemacht. Für Fehler im Text oder Code bin ich verantwortlich, auch bei Übernahme aus generativen KI-Systemen. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Kaiserslautern, den 7.10.2026

Mihir Mahesh Ambli

Abstract

English

Zusammenfassung

Deutsche

Contents

1 Introduction **1**
- 1.1 Motivation . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Research Questions . . . 3
- 1.4 Approach . . . 5
- 1.5 Thesis Outline . . . 5

2 Background **7**
- 2.1 The Web and HTTP . . . 7
- 2.2 Cyber Deception, Honeypots and Honeytokens . . . 8
- 2.3 SQL Injection . . . 9
- 2.4 Proxies and Reverse Proxies . . . 9
- 2.5 Rewriting HTTP Responses . . . 10
- 2.6 Extension Mechanisms . . . 11
- 2.7 Extension Runtimes . . . 11
- 2.8 Measuring Performance . . . 12

3 Related Work **14**

4 Design of the Deception Module **16**
- 4.1 Requirements and Threat Model . . . 16
- 4.2 Architecture . . . 17
- 4.3 Honeytoken Types . . . 17
- 4.4 The Fake SQL-Injection Trap . . . 18
- 4.5 Per-Request Processing . . . 19

5 Implementation on Three Reverse Proxies **21**
- 5.1 Common Structure . . . 21
- 5.2 OpenResty (Reference Variant) . . . 22
- 5.3 Envoy with a Lua Filter . . . 22
- 5.4 Envoy with a WebAssembly Filter . . . 22
- 5.5 Apache httpd with mod_lua . . . 23
- 5.6 Levelling the Variants . . . 23
- 5.7 Verifying Equivalence . . . 24

6 Evaluation Methodology **26**
- 6.1 Goals and Metrics . . . 26

6.2 Testbed . . . 26
6.3 Workload . . . 26
6.4 Tiers and Pairing . . . 26
6.5 Measurement Planes . . . 27
6.6 Run Validity . . . 27

7 Results **28**
7.1 RQ1: Latency Added . . . 28
7.1.1 Median Latency per GET across Rates . . . 28
7.1.2 Latency above the Origin Server . . . 28
7.1.3 Tail Latency . . . 28
7.1.4 Request Position and Reconnects . . . 28
7.1.5 Answer to RQ1 . . . 29
7.2 RQ2: CPU and Capacity . . . 29
7.2.1 CPU per Request . . . 29
7.2.2 Cost Attribution . . . 29
7.2.3 The Cost of Measuring . . . 29
7.2.4 Per-Mechanism Cost and Page Size . . . 29
7.2.5 Answer to RQ2 . . . 29
7.3 RQ3: Functional Correctness . . . 30
7.3.1 Response Integrity under Load . . . 30
7.3.2 Detection Recall and False Positives . . . 30
7.3.3 Answer to RQ3 . . . 30
7.4 RQ4: Influence of the Extension Runtime . . . 30
7.4.1 Per-Type Operation Cost . . . 30
7.4.2 The SQLi Trap: Scan Depth versus Decoding . . . 30
7.4.3 Stability of Apache mod_lua . . . 30
7.4.4 Answer to RQ4 . . . 31

8 Discussion **32**
8.1 Synthesis across Research Questions . . . 32
8.2 Choosing a Variant for Deception . . . 32
8.3 Where the Cost Comes From . . . 32
8.4 Deployment Implications . . . 32
8.5 Threats to Validity . . . 32
8.6 Limitations . . . 32

9 Conclusion and Future Work **33**
9.1 Summary . . . 33
9.2 Future Work . . . 33

A Configuration Reference **34**

B Parity-Check Request Set **35**

C Reproducing the Benchmarks **36**

D Additional Result Tables **37**

1 Introduction

Many services today are offered through web applications, which any client on the internet can reach. This reachability makes them a target. In the incidents analysed for Verizon's 2026 Data Breach Investigations Report, exploiting a software vulnerability was the most common way into an organisation: 31 % of breaches started this way, more than started with stolen passwords [1]. Protecting software that is exposed to the internet is therefore one of the central tasks of security.

Traditional security measures aim to keep attackers out and to notice those who get in: firewalls, authentication and intrusion prevention at the boundary of a network, and intrusion detection systems (IDS) that watch for suspicious activity. These measures are essential, but they have well-known limits. Zhang and Thing observe that such perimeter defences have proven weak against infiltration, that targeted attacks often exploit previously unknown (zero-day) vulnerabilities and leave few traces, and that intrusion detection systems tend to raise an overwhelming number of false alarms [2]. Axelsson showed that the rate of false alarms, rather than the ability to recognise intrusions, is what limits the effectiveness of intrusion detection [3]. The balance also favours the attacker: the defender has to protect every point of a system, whereas the attacker needs only one vulnerability [2].

Active defence responds to these limits by acting against the attacker directly. Denning defines active cyber defence as direct defensive action taken against specific threats, in contrast to passive defence, which makes systems more resilient to attack in general [4]. Deception is one form of active defence [4, 2]. Instead of only blocking what an attacker does, it acts on what the attacker perceives: it places fake but plausible elements in a system in order to mislead the attacker, make the attacker waste effort, and increase the attacker's risk of being detected [2]. Because legitimate users have no reason to interact with these elements, an interaction with one of them is a strong sign of an attack. Deception can therefore detect attacks with few false alarms, including attacks that exploit unknown vulnerabilities, and it serves as an additional layer of defence alongside the measures above [2].

This thesis applies deception to web applications. It places deceptive elements in the traffic between a web application and its visitors, and it measures what doing so costs. The rest of this chapter explains why web applications are a suitable place for deception and how it can be deployed there (Section 1.1), which problem this raises (Section 1.2), the questions the thesis answers (Section 1.3), how it answers them (Section 1.4), and how the thesis is organised (Section 1.5).

1.1 Motivation

A web application reveals more to its visitors than the pages it displays. The source code of its pages in the HyperText Markup Language (HTML), the headers and cookies of its responses in the Hypertext Transfer Protocol (HTTP), and auxiliary files such as `robots.txt` are delivered to every client, including one that is preparing an attack (Section 2.1 recalls these elements). Security testing of a web application accordingly begins with information gathering. The Web Security Testing Guide of OWASP, the Open Worldwide Application Security Project [5], instructs a tester to read the comments and metadata in a page's HTML source, which may reveal internal information, and the `robots.txt` file, which may list paths that the site does not link to [6]. An attacker can read the same artefacts.

This first phase of an attack is where deception can act. A *honeytoken* is a piece of fake data, such as a fake credential or a hidden link, that no legitimate user has a reason to use (Section 2.2). Planted in the responses of a web application, honeytokens are visible to any client that studies those responses closely, and their use raises an alert. Han et al. deployed such deceptive elements in a production web application for seven months and recorded no false alarm [7]. In a red-team exercise in the same study, the deceptive elements detected many attackers but not all, and the authors suggested deception as a companion to other detection techniques rather than a standalone protection [7].

The practical obstacle is deployment. Kahlhofer and Rass observe that the adoption of cyber deception is held back by the difficulty of deploying it in real-world software systems, and that the operator responsible for security usually has the built application but not its source code [8]. Among the methods for deploying deception without changing the application, placing a *reverse proxy* in front of it, a server that receives requests on behalf of the application and forwards them to it (Section 2.4), is the one most commonly found in research [8]; Chapter 3 reviews such systems. Widely used servers already fill this role and can run custom code. The web servers nginx and Apache HTTP Server (httpd) act as reverse proxies through their proxy modules and can be extended through further modules [9, 10, 11, 12], and Envoy, a dedicated proxy, can be extended with filters written in C++, Lua or WebAssembly [13, 14]. Placing honeytokens in a reverse proxy that the operator already runs therefore requires no change to the application.

1.2 Problem Statement

Running deception logic on every request is not free. Kahlhofer and Rass note that the reverse-proxy pattern often comes with a significant performance overhead and call for research to reduce it [8], and Zhang and Thing call for the cost of deception to be quantified [2]. The available measurements, however, concern proxies running

generic or empty extensions, and the deception systems built so far report little about their own cost (Chapter 3). What the work of deception itself costs, which is searching each request for planted values and rewriting response bodies, is therefore unknown, and so is how that cost differs between proxies and runtimes and how it compares with the cost of the proxy itself.

Rewriting responses also risks breaking them. HTTP describes a response body through mechanisms such as its declared length and its compression, and inserting honeytokens can invalidate them (Section 2.5).

The problem addressed by this thesis is therefore the following. An operator who wants to deploy honeytokens in a reverse proxy has no measured basis for choosing the proxy and the runtime that executes the deception code, nor for estimating what the deception costs in latency, central processing unit (CPU) time, capacity and functional correctness.

To answer this question, the thesis develops the *Reverse Proxy Deception Module* (RPDM), a deception mechanism that runs inside an existing reverse proxy, referred to as the *host proxy*, which forwards requests to the *origin server* that serves the application. RPDM is implemented in four *variants* on three reverse proxies, each variant being one combination of host proxy and the runtime that executes the code of RPDM:

- OpenResty, a web platform built on nginx [15], where RPDM is Lua code run by nginx's Lua module on LuaJIT, a just-in-time (JIT) compiler for Lua;
- Envoy, where RPDM is an HTTP filter written in Lua and run by LuaJIT;
- Envoy, where RPDM is an HTTP filter compiled to WebAssembly and run by the V8 engine;
- Apache httpd, where RPDM is Lua code run by the module `mod_lua` on the standard Lua interpreter, called PUC Lua in this thesis after its developers at PUC-Rio, the Pontifical Catholic University of Rio de Janeiro [16].

Sections 2.6 and 2.7 explain these proxies and runtimes.

The variants were measured in a controlled testbed. All measurements were taken on a single virtualised host with an origin server that serves static pages, so the comparison between variants is the claim the results support, while absolute values are not expected to carry over to other hardware. The effectiveness of honeytokens against real attackers is outside the scope of this work; detection is tested only against scripted requests whose expected alerts are known.

1.3 Research Questions

The thesis addresses four research questions (RQ1 to RQ4), each divided into subquestions. Chapter 7 answers them in this order.

RQ1 How much latency does RPDM add to a request in each variant, how does it compare with the latency the host proxy itself adds over direct access to the origin server, and how do both change with load?
- RQ1.1 How much does RPDM add to the median and to the 95th-percentile latency of a request in each variant?
- RQ1.2 How does this compare with the latency that the host proxy itself adds over direct access to the origin server?
- RQ1.3 How do both change as the load increases?

RQ2 How much CPU time and capacity, the highest request rate a variant sustains, does RPDM cost in each variant, and where does that cost arise?
- RQ2.1 How much CPU time per request does RPDM add in each variant?
- RQ2.2 How much does RPDM reduce the capacity of each variant?
- RQ2.3 Where does the added cost arise: in the detection and insertion code of RPDM or in the work around it, for which requests, and how does it grow with the size of a page?

RQ3 Apart from the honeytokens it inserts, does RPDM leave the responses of the host proxy unchanged, and does it raise every alert it is expected to raise?
- RQ3.1 For the kinds of response a web site serves, such as compressed pages, partial content and responses to `HEAD` requests, does each variant return the same response with RPDM as without it?
- RQ3.2 Do the responses remain intact under load?
- RQ3.3 Does every variant raise every expected alert, and does none raise an alert without RPDM?

RQ4 To what extent do the answers to RQ1–RQ3 depend on the extension runtime: LuaJIT, PUC Lua or WebAssembly?
- RQ4.1 How does the cost of the operations for each honeytoken type differ between the runtimes?
- RQ4.2 In the SQL-injection trap, what do decoding the input and scanning the signatures cost in each runtime?

Not all of these questions were planned from the start. The thesis proposal asked how the extension modules of a web server could implement a deception mechanism that serves hidden honeytokens to every client and flags a client only when it uses one, and how the honeytokens could be inserted without unacceptable latency and without breaking the application . The first part is answered by the design and implementation of RPDM (Chapters 4 and 5), which were extended from nginx to three reverse proxies; the concerns about latency and functionality became RQ1 and RQ3. The other questions arose during the work. Latency alone did not capture the cost: at some load levels, the difference in latency was lost in the noise of the testbed, while the CPU time per request remained measurable, and the highest sustainable request rate required an experiment of its own (RQ2). The influence of the runtime (RQ4) became a question when the SQL-injection trap showed that scanning a longer list of signatures cost measurably more only in the Apache variant, the one whose code is interpreted; a measurement of a single variant would have missed this. Finally,

the comparison with direct access to the origin server (RQ1.2) was added when it became clear that the earlier measurements showed what RPDM costs on top of a reverse proxy, but not what the reverse proxy itself costs.

1.4 Approach

RPDM plants five types of honeytoken in the responses of the host proxy: HTML comments, HTTP response headers, cookies, decoy paths and hidden form fields. It also answers a fake login endpoint itself, replying to attempts at SQL (Structured Query Language) injection with a fabricated database error. When a request triggers a honeytoken, RPDM writes an alert and records the client's address; if the trigger is the keyword of an HTML-comment honeytoken, RPDM also removes it from the query string before the request reaches the origin server. A single configuration file drives all four variants.

To compare the variants fairly, they were first made to behave identically. An automated check sends the same requests to every variant and compares the responses, the alerts and the requests that reach the origin server; the remaining differences are documented (Chapter 5). Each variant was then measured at three levels of load in three configurations: requests sent directly to the origin server without any proxy, through the host proxy without RPDM, and through the host proxy with RPDM. The three were measured minutes apart within the same run, so that the differences between them can be attributed to the host proxy and to RPDM respectively (Chapter 6). The measurements record the latency a client experiences, the CPU time the host proxy consumes, and whether the responses with RPDM differ from those without it.

1.5 Thesis Outline

Chapter 2 refreshes the background that the rest of the thesis relies on: the elements of HTTP and HTML that honeytokens use, cyber deception and honeytokens, SQL injection, proxies and in particular reverse proxies, the HTTP mechanisms that rewriting a response can break, how the three reverse proxies run extensions, the runtimes that execute them, and the basics of measuring performance. Chapter 3 reviews earlier deception systems placed in front of web applications, what is known about their cost, and the pitfalls of performance benchmarking. Chapter 4 presents the design of RPDM independently of any host proxy, and Chapter 5 describes the four variants and how they were made to behave identically. Chapter 6 describes how the variants are measured, and Chapter 7 presents the results, one section per research question. Chapter 8 discusses the results, the choice of a variant for deception, the threats to validity and the limitations, and Chapter 9 concludes and outlines future

work. The appendices contain the configuration reference, the parity-check request set, instructions for reproducing the benchmarks and additional result tables.

2 Background

This chapter refreshes the concepts that the rest of the thesis relies on. Most of them are familiar to a reader with a background in networking and security; they are collected here so that later chapters can refer to them. The chapter recalls the parts of the web and of HTTP that honeytokens use, cyber deception and honeytokens, SQL injection, proxies and in particular reverse proxies, the HTTP mechanisms that rewriting a response can break, how the three reverse proxies of this thesis run extension code and which runtimes execute it, and the basic quantities with which performance is measured.

2.1 The Web and HTTP

A web application is served by an *origin server* and used through a client, usually a web browser, which HTTP calls a *user agent* [17]. Client and server exchange HTTP messages. A request consists of control data, which names the method, the request target and the protocol version, followed by header fields and, optionally, content, also called the body; a response carries a status code in place of the method and target [17]. Three methods matter in this thesis: `GET` requests a resource; `POST` submits data, such as the fields of an HTML form, for the resource to process; and `HEAD` is identical to `GET` except that the response carries no body [17]. Status codes are grouped into classes, among them 2xx for success, 4xx for an error on the client's side, such as a request for a resource that does not exist (404), and 5xx for an error on the server's side [17]. Header fields carry further information; for example, `Host` names the host that the client addressed, and `Content-Type` the media type of the body, such as an HTML page [17].

The address of a resource is its Uniform Resource Locator (URL), a form of Uniform Resource Identifier [18]. In a request to an origin server, the request target is the path and the query of the URL [19]. The query begins after the first question mark [18]. It usually holds parameters of the form `name=value` separated by `&`, the format in which a browser submits a form [20]. A character that is not allowed in a URL, or that would act as a delimiter, is percent-encoded as `%` followed by two hexadecimal digits; `%20`, for example, encodes a space [18].

Cookies let a server keep state across requests. The server sends name–value pairs to the client in a `Set-Cookie` header, and the client returns them in a `Cookie` header with later requests to that server [21].

An HTML page can also contain content that a visitor does not normally see: comments, delimited by `<!-` and `->` [22], and hidden input fields, which represent a value that is not intended to be examined or manipulated by the user [23]. When a form is submitted with `GET`, its fields, hidden ones included, are appended to the URL as its query; with `POST`, they are sent in the body [20]. Finally, many sites publish a file `robots.txt` that tells web crawlers which paths not to visit. It is not an access control, and the paths it lists can point a tester to areas worth examining [6].

2.2 Cyber Deception, Honeypots and Honeytokens

Cyber deception, introduced in Chapter 1, defends a system with fake but plausible elements that mislead an attacker and reveal the attacker's presence [2]. Han et al. classify deception techniques by the layer at which they act: the network, a system, an application, or its data [24]. The deceptive elements of this thesis act at the application layer: they are part of the HTTP traffic of a web application.

The two best-known forms of deception are honeypots and honeytokens. Spitzner defines a honeypot as an information system resource whose value lies in unauthorised or illicit use of that resource [25]; a typical honeypot is a decoy system that lures attacks away from production systems so that they can be monitored [4]. A honeytoken applies the same definition to data: a fake credential, document or URL parameter that no legitimate workflow uses [24]. Honeytokens are simple and cheap to deploy, and they can take many forms [2]. Because a honeytoken has no legitimate use, its use is the detection signal.

In a web application, honeytokens are placed in the artefacts that a reconnoitring client reads (Section 2.1). The OWASP AppSensor project, as summarised by Han et al. [7], distinguishes three detection signals:

- *alteration of honey trap data*: a client modifies a value the server planted, such as a hidden form field, an additional URL parameter or a cookie;
- *request for a honey trap resource*: a client requests a page or directory that is advertised only to readers of, for example, `robots.txt`;
- *use of collected honey trap data*: a client uses information that is visible only in the HTML source, such as a fake account.

The three signals differ in what the detector must compare. Alteration requires the planted value, a resource request requires only the request path, and data use requires a search of incoming requests for the planted value. Chapter 4 maps each of the five honeytoken types of RPDM (Section 1.4) onto one of these signals.

2.3 SQL Injection

Many web applications store their data in a database and query it with the Structured Query Language (SQL). SQL injection exploits an application that builds such a query from user input: input that contains SQL syntax can change the structure of the query [6]. It is tested by sending characters and clauses that would have this effect. The OWASP testing guide lists a single quote or semicolon as the usual first probe, comment delimiters and the keywords `AND` and `OR` to alter the query logic, and always-true conditions such as `OR 1=1` that let a login succeed without valid credentials [6]. Detailed database error messages help the tester reconstruct the original query and refine the payload; when they are hidden, the tester must fall back on blind techniques [6].

An endpoint that answers such probes with a plausible database error is therefore an attractive target for a follow-up attack. A request that contains no SQL syntax does not receive the error. The fake login endpoint of RPDM (Section 1.4) uses this property.

2.4 Proxies and Reverse Proxies

HTTP allows intermediaries between the client and the origin server. Request for Comments (RFC) 9110, the standard that specifies the semantics of HTTP, names three common forms [17]:

- a *proxy*, often called a forward proxy, is chosen by the client, usually through local configuration, and forwards the client's requests, for example to route an organisation's requests through a common intermediary for security services or shared caching;
- a *gateway*, also called a *reverse proxy*, acts as an origin server towards the client but forwards the requests it receives to one or more inbound servers, for example to shield legacy services, to cache responses or to balance load across machines;
- a *tunnel* relays a connection blindly, without changing the messages.

In addition, an interception proxy, also known as a transparent proxy, redirects traffic without being chosen by the client [17].

A reverse proxy appears to the client like an ordinary web server: the client needs no special configuration, and the reverse proxy returns content as if it were the origin itself [10]. Because it terminates the client's request and produces the response the client receives, a reverse proxy can inspect and modify both. This is the position in which RPDM runs.

Two kinds of software fill this role in this thesis. Web servers such as nginx and Apache httpd act as reverse proxies through a proxy module: nginx describes itself

as an HTTP web server, reverse proxy, content cache and load balancer, among other roles [26], and its module `ngx_http_proxy_module` passes requests to another server [9]; in Apache httpd, `mod_proxy` implements a proxy and gateway [10]. Envoy, by contrast, is a dedicated proxy for the application layer (layer 7), designed as a self-contained process that runs alongside application servers [13]. Section 2.6 shows that this difference also determines how the two kinds are extended.

Whichever kind it is, a reverse proxy handles each exchange in two phases into which extension code can be inserted: a request phase, before the request is forwarded, and a response phase, in which first the response headers and then the response body pass through the proxy [27, 11]. RPDM looks for triggered honeytokens in the request phase and inserts honeytokens in the response phase. Two properties of these phases matter for its cost:

- *Streaming versus buffering.* A response body can arrive in several pieces. An extension can modify each piece as it passes, or wait until the whole body has arrived; waiting makes it simpler to insert text at a particular position but requires holding the whole body in memory.
- *Connection reuse.* HTTP/1.1 connections are persistent by default, so several requests can share one connection [19]. Whether a proxy reuses its connections to the origin server depends on its configuration; Section 5.6 shows that this affected the comparison in this thesis.

2.5 Rewriting HTTP Responses

Inserting text into a response body changes the message, while some HTTP mechanisms still describe the body that the origin server sent. RQ3 checks whether RPDM breaks them (Chapter 7):

- *Length.* `Content-Length` states the length of the body and marks where the message ends [17]. A proxy that lengthens the body must correct this header or remove it and send the body in chunks of stated length instead [19].
- *Compression.* A body compressed as declared by `Content-Encoding`, for example with `gzip`, must be decompressed before it can be read [17]. Text inserted into the compressed bytes makes the body undecodable.
- *Partial content.* A range request asks for part of a resource, for example to resume an interrupted download, and is answered with status 206 [17]. The client joins this part to the parts it already holds, so a modified part corrupts the result.
- *HEAD.* A response to `HEAD` has no body but should carry the same headers as the response to the corresponding `GET` [17].

2.6 Extension Mechanisms

The two kinds of reverse proxy from Section 2.4 are extended in different ways. In the web servers, extension code runs through a *server module* that hooks into the server's request processing: OpenResty builds the `lua-nginx-module` into nginx [11], and Apache httpd provides `mod_lua` [12]. In both, RPDM is Lua code that the module runs. In Envoy, extension code runs as an *HTTP filter*, which is plugged into the proxy's handling of HTTP connections [13]; Envoy offers a filter that runs Lua code and a filter that runs code compiled to WebAssembly [28, 29]. Three properties of these mechanisms matter for the results that follow.

First, the proxies hand the response body to the extension in different ways. OpenResty passes each piece of the body to the extension as it arrives, together with a flag that marks the last piece [11]. In Envoy's Lua filter, the call that reads the body suspends the extension until the whole body has been buffered, and the amount buffered is bounded by the buffer limits of the proxy [28].

Second, the lifetime of the interpreter state that runs the extension differs. In Apache, the setting `LuaScope` selects it, from a new state for every use to one state per thread or a shared pool of states; the documentation states that reusing a state per thread or from a pool is roughly two to three times faster than creating a new one per request [12]. The Apache variant of RPDM reuses one state per thread, a choice that Chapter 7 returns to when it discusses the stability of Apache.

Third, Envoy's WebAssembly filter, which its documentation marks as experimental [29], loads a WebAssembly module that implements the Proxy-Wasm interface, a proxy-independent interface between a proxy and WebAssembly extensions [30]. Envoy can execute such modules on several WebAssembly runtimes; this thesis uses V8 [31].

2.7 Extension Runtimes

RQ4 asks how much of the cost of RPDM depends on the runtime that executes it. The four variants use three runtimes.

PUC Lua. The reference implementation of Lua is developed at PUC-Rio, the Pontifical Catholic University of Rio de Janeiro [16]. It translates Lua into instructions for a virtual machine and executes them one at a time in an interpreter loop [32]. The Apache variant runs on PUC Lua 5.1.

LuaJIT. LuaJIT is a just-in-time (JIT) compiler for Lua [33]. It starts by interpreting the code; a loop or function becomes a candidate for compilation to machine code

once it has been detected as hot, by default after 56 iterations [34]. This is one reason why each variant is warmed up before it is measured (Chapter 6).

WebAssembly. WebAssembly is a portable bytecode designed for safe execution [35]. A WebAssembly module works on its own memory and reaches anything outside it only through functions that its host provides [35]. In a proxy, every read or write of request or response data is therefore a call from the module into the proxy.

The four variants are arranged so that RQ4 can separate the effect of the runtime from that of the proxy. The two Envoy variants share a proxy but differ in runtime, and the OpenResty and Envoy Lua variants share LuaJIT but differ in proxy. The Apache variant is the only one whose extension code is always interpreted; because it differs from the others in both proxy and runtime, a difference in its results cannot be attributed to either alone.

2.8 Measuring Performance

The evaluation in Chapters 6 and 7 uses a few standard quantities.

Latency. Latency is the time a client waits from sending a request until it has received the response. It varies from request to request and is therefore described by its distribution. The median, or 50th percentile (p50), is the latency below which half of the requests stay; the 95th percentile (p95) is the latency below which 95 % of them stay. The median describes a typical request, whereas high percentiles describe the slowest requests, the *tail*; for questions such as worst-case latency, a mean or median is not sufficient and percentiles are needed [36].

Throughput and capacity. Throughput is the number of requests served per second. The capacity of a system is the highest request rate it can sustain (RQ2).

CPU time. CPU time is the time that processors spend executing a program, divided into user time, spent in the program's own code, and system time, spent in the operating system kernel on the program's behalf. Unlike latency, it does not include time spent waiting, and it therefore measures the work a component performs.

Containers and control groups. A container is a runnable instance of an image, isolated from other containers through namespaces [37]. A control group (cgroup) is a Linux kernel mechanism that organises processes hierarchically and distributes

system resources among them; its file `cpu.stat` reports the CPU time consumed by all processes in the group, in total and divided into user and system time [38]. In the testbed of this thesis, every component runs in its own container, and the benchmark reads the CPU time of each container's control group (Chapter 6).

3 Related Work

Chapter 1 argued that a reverse proxy is the most common way in research to add deception to a web application without changing it, and that the cost of doing so is largely unknown. This chapter reviews the systems that take this approach, what is known about their cost, and the pitfalls of measuring such costs.

Han et al. built a reverse proxy that adds deceptive elements, such as fake cookies, hidden form fields and fake vulnerabilities, to the responses of a web application and removes them from incoming requests [7]. One of its fake vulnerabilities answers an SQL-injection probe with a fabricated MySQL error [7], the same principle as the fake login endpoint of RPDM (Section 2.3). SunDEW combines deceptive elements inside the application with a reverse proxy [39], and Cloxy places a reverse proxy in front of legacy web applications that cannot easily be updated [40]. Koney deploys traps in Kubernetes clusters and modifies response bodies through an Envoy extension compiled to WebAssembly [41], the same kind of deployment as the Envoy WebAssembly variant of RPDM. Honey Infiltrator shows an alternative to a proxy: a network bridge that injects honeytokens into passing web traffic [42].

Little is known about what these systems cost. The authors of Cloxy report that their prototype increases the delay of communication with the web application, within bounds they consider tolerable [40]. The authors of Koney report no measurement of its cost and note that reverse proxies often add a significant latency overhead [41]. The available measurements come from service meshes, which place a proxy beside every service of an application [8]. In such a setting, Zhu et al. found that even an Envoy filter that does nothing has a substantial overhead, and that such a filter written in Lua cost about three times as much latency as one compiled to WebAssembly [14]. A filter that does nothing, however, performs no deception work, and the study does not include OpenResty or Apache httpd.

Performance comparisons of this kind are easy to get wrong. Van der Kouwe et al. found benchmarking flaws in all but one of the 50 systems-security papers they surveyed [43], and Mytkowicz et al. showed that a seemingly innocuous change to the experimental setup can lead to an incorrect conclusion [44]. An earlier version of the benchmark in this thesis produced such errors itself. Chapter 6 describes the errors that were found, the measures that replaced them, and the limitations that remain.

Each of the deception systems above was built on a single interception platform, and the overhead studies run no deception logic. None of them therefore shows how the cost of one deception mechanism differs between proxies and runtimes. This thesis

addresses that gap: RQ1 measures the latency RPDM adds in each variant and sets it against the latency of the host proxy itself, RQ2 measures its CPU and capacity cost, RQ3 checks whether it leaves the responses of the host proxy unchanged and raises every expected alert, and RQ4 relates the differences to the extension runtime.

4 Design of the Deception Module

This chapter describes RPDM independently of any host proxy: the setting it is designed for, where it sits between the client and the origin server, the honeytokens it plants, its fake login endpoint, and the steps it performs on every request. Chapter 5 then shows how the four variants realise this design.

4.1 Requirements and Threat Model

RPDM is designed for the setting described in Section 1.1: an operator runs a reverse proxy in front of a web application whose source code it cannot change [8]. The attacker is a client that reconnoitres the application. It reads the responses, including their HTML source, headers and cookies, and sends requests of its own, for example by following hidden links, editing cookies and form fields, or probing for SQL injection (Sections 2.2 and 2.3). A legitimate user, by contrast, uses a browser, which returns cookies and hidden form fields unchanged.

Five requirements follow from this setting:

R1 *No change to the application.* RPDM runs entirely inside the host proxy; the origin server is not modified.

R2 *Planted secrets do not reach the application.* When a client sends back the trigger keyword of an HTML-comment honeytoken, the keyword is removed before the request is forwarded.

R3 *Attacker input cannot corrupt RPDM's output.* Line breaks in attacker-controlled values are replaced before an alert is written, so that an attacker cannot forge alert lines, and input that the fake login endpoint reflects is HTML-escaped, so that the trap does not itself become a cross-site scripting vector.

R4 *One configuration for all variants.* The honeytokens and the trap are defined once, in one configuration file that every variant reads.

R5 *Identical behaviour in every variant.* For the same request, every variant returns the same response, writes the same alerts and forwards the same request to the origin server. Otherwise, a difference in cost between two variants could reflect a difference in work rather than in proxy or runtime (Chapter 5).

RPDM detects and records; it does not block or slow down an attacker. The client addresses it records are not used to change any response. Its fake login endpoint

does not authenticate anyone, and the effectiveness of its honeytokens against real attackers is outside the scope of this thesis (Section 1.2).

4.2 Architecture

Figure 4.1 shows where RPDM sits. Every request passes through the host proxy, which forwards it to the origin server and returns the origin's response to the client. RPDM acts in the two phases introduced in Section 2.4: in the request phase, it checks the request for triggered honeytokens; in the response phase, it inserts honeytokens into the response. It reads its configuration when it is loaded, not on every request. Requests to the fake login endpoint are the exception: the host proxy answers them itself, and they never reach the origin server (Section 4.4).

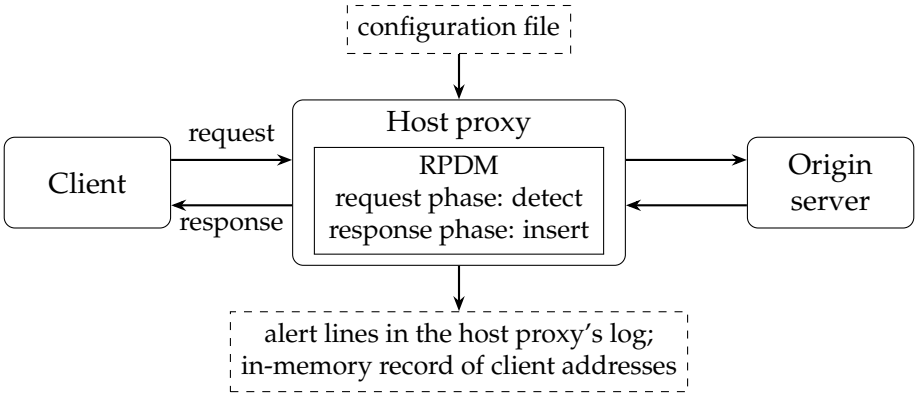


Figure 4.1: RPDM inside the host proxy, between the client and the origin server.

RPDM has two outputs. It writes an alert line to the log of the host proxy for every triggered honeytoken, and it records the address of the client in memory. The alert lines begin with the fixed prefix `WADM ALERT`; *WADM* is the name the implementation uses internally. These lines are what RQ3 counts when it checks whether every expected alert was raised.

4.3 Honeytoken Types

RPDM plants the five types of honeytoken listed in Table 4.1. Each is detected through one of the three signals introduced in Section 2.2.

Each honeytoken applies either to every path or to a list of exact paths, as set in the configuration. Headers and cookies are added to every response for a matching path, whatever its content type. Comments, links and form fields are inserted only into responses whose content type is HTML. RPDM does not check the status code or the

Type	Planted as	Detected when	Signal
HTML comment	a comment before `</body>`, or at the end of the body if there is none	its trigger keyword appears in the query string	use of collected data
HTTP header	a decoy response header	its trigger keyword, the header's value, appears in the path, `Host` header or query string of a later request	use of collected data
Cookie	a `Set-Cookie` header with a fixed value	the cookie comes back with a different value	alteration of trap data
Decoy path	a hidden link before `</body>`, pointing to a path the application does not use	a request for that path or, if so configured, for a path below it	request for a trap resource
Form field	a hidden input field before `</form>`	the field is submitted with a different value	alteration of trap data

Table 4.1: The five honeytoken types of RPDM and the signal by which each is detected.

content coding of a response before inserting into it; Chapter 7 shows what these rules mean for the HTTP mechanisms of Section 2.5.

The detectors examine the request line and headers. RPDM can also inspect the bodies of form submissions, but this option is disabled in all experiments of this thesis and is implemented in only two of the four variants.

4.4 The Fake SQL-Injection Trap

In addition to the honeytokens, RPDM contains a fake login endpoint that imitates an application vulnerable to SQL injection, following the fake vulnerabilities of Han et al. (Chapter 3). The endpoint, `POST /api/login`, does not exist on the origin server; the host proxy answers it itself. Requests with other methods to the same path are forwarded as usual.

The trap reads the watched form fields of the request body, `username` and then `password`. It decodes each value, converts it to lower case and collapses whitespace, and then searches it for a list of 22 literal signatures, such as `' or`, `union select` and `sleep(`, which correspond to the probes described in Section 2.3. The signatures are matched as plain substrings rather than as regular expressions, which are not available alike in all four runtimes.

- *Signature found.* RPDM writes an alert, records the client address and returns status 500 with a page that imitates a MySQL syntax error. The page reflects the attacker's input, HTML-escaped and truncated to 200 characters, inside a plausible `SELECT` statement.
- *No signature found.* RPDM returns status 401 with a "sign in failed" page, so that the endpoint behaves like a real login. Because the request never reaches the origin server, RPDM records it with a separate log line, prefixed `WADM TRAP`.

No honeytokens are inserted into either page. In one variant, the host proxy cannot answer the request on its own and replaces the origin server's reply instead; the client receives the same response, and Chapter 5 explains why.

4.5 Per-Request Processing

Everything that depends only on the configuration is computed once, when the configuration is loaded: for every configured path, RPDM prepares a plan that holds the comments, headers, cookies and markup to insert into responses for that path. For each request, RPDM then performs the following steps, which Figure 4.2 summarises.

1. *Setup.* The client address is taken from the network connection, not from a header that the client could set. The path is the request target up to the first `?`, used without decoding. The query string is split once into its parameters, in their original order; a parameter is decoded only if it contains an encoded character.
2. *Trap.* A `POST` request to `/api/login` is handed to the trap (Section 4.4), and processing ends there.
3. *Detection.* The detectors for headers, cookies, decoy paths and form fields run in a fixed order. RPDM then searches the decoded query parameters for the trigger keywords of the HTML comments. A parameter that contains a keyword is removed, and the query string is rebuilt from the original text of the remaining parameters, so that their order and encoding are kept (R2).
4. *Recording.* If any honeytoken was triggered, RPDM records the client address and writes one alert line per triggered honeytoken, with line breaks in attacker-controlled values replaced (R3).
5. *Response.* RPDM looks up the plan for the path, sets the decoy headers and adds the cookies. If the response is HTML and the plan inserts into the body, RPDM removes the `Content-Length` header, because the length of the body changes (Section 2.5), waits until the whole body has arrived (Section 2.4), and inserts each honeytoken at its anchor, found by a plain, case-sensitive search.

These steps are the specification that every variant implements. Chapter 5 describes how each of the three reverse proxies realises them, and how it was verified that the four variants behave identically (R5).

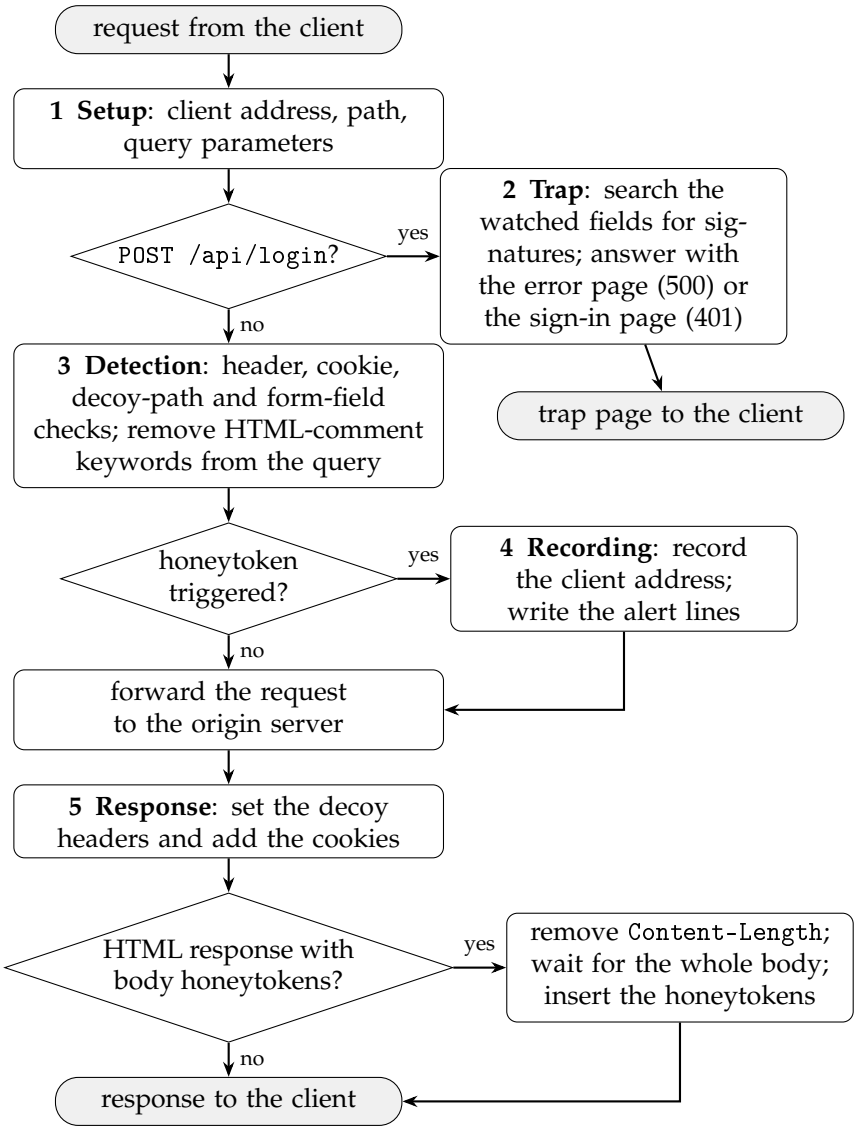


Figure 4.2: Per-request processing of RPDM. Steps 1 to 4 run in the request phase, step 5 in the response phase (Section 2.4).

5 Implementation on Three Reverse Proxies

Chapter 4 specified RPDM independently of any host proxy. This chapter describes how the four variants implement that specification and how they were made comparable. Section 5.1 gives the structure that all four share, Sections 5.2–5.5 describe only what is particular to each variant, Section 5.6 describes how the variants were levelled, and Section 5.7 describes the check that they behave identically.

5.1 Common Structure

All four variants read the same configuration file (R4) and implement the five steps of Section 4.5 in the same order. In OpenResty and Apache httpd, RPDM is Lua code run by the server's Lua module; in Envoy, it is an HTTP filter (Section 2.6). Each compiles the configuration into its per-path plans when it loads it, checks the request in the hook that the host proxy runs before the request is forwarded, and inserts the honeytokens in the hooks that see the response. Table 5.1 lists these hooks and the lifetime of the interpreter state that runs RPDM in each variant (Section 2.6).

Variant	Request phase	Response phase	Interpreter state
OpenResty	`access_by_lua`	`header_filter_by_lua`, `body_filter_by_lua`	one Lua virtual machine per worker process [11]
Envoy, Lua	`envoy_on_request`	`envoy_on_response`	one Lua state per worker thread [28]
Envoy, WebAssembly	request-header and request-body callbacks	response-header and response-body callbacks	one module instance per worker thread [31]
Apache httpd	`LuaHook AccessChecker`; `LuaMapHandler` for the trap	`LuaOutputFilter` for the body	one Lua state per thread and script file

Table 5.1: Where RPDM runs in each variant.

Every variant also contains timers around its detection and insertion steps. They write their measurements to the log of the host proxy when the configuration enables them; this is the only setting in which the configuration measured as deployed differs from the one used for timing (Chapter 6).

5.2 OpenResty (Reference Variant)

In OpenResty, RPDM is Lua code that the `lua-nginx-module` runs. It compiles the configuration once, before nginx starts its worker processes, so that every worker inherits the compiled plans. The client addresses that RPDM records are kept in a shared memory zone that all worker processes use [11]. Because nginx passes the response body to the extension in pieces (Section 2.6), the body filter collects the pieces and inserts the honeytokens when the last one has arrived. The other three variants were aligned to this implementation, and it is the reference against which they are compared in Section 5.7.

5.3 Envoy with a Lua Filter

This variant runs RPDM as an Envoy HTTP filter written in Lua. Each worker thread loads and compiles the configuration in its own Lua state. In the response phase, the filter waits until Envoy has buffered the whole body, within Envoy's buffer limits (Section 2.6), and writes the modified body back. Because Envoy then knows the final length, this is the only variant that sends a `Content-Length` header with an injected page; the other three send it without one (Section 2.5). The trap is answered by the filter itself.

5.4 Envoy with a WebAssembly Filter

This variant implements RPDM in Rust against the Proxy-Wasm interface and compiles it to a WebAssembly module, which Envoy runs on V8 (Section 2.6). Envoy passes the configuration file to the module at start-up, and the module compiles it once. Because every read of a request header is a call from the module into Envoy (Section 2.7), the module reads each header at most once and only when it is needed: the method only for the trap's path, and the cookie header only if a cookie honeytoken exists. In the response phase it lets the headers continue before the body arrives, removing their `Content-Length` when the page will be injected, and then waits for the whole body and inserts the honeytokens into its bytes.

The trap cannot be implemented here as on the other variants. In Envoy 1.30, the module can answer a request itself only from the callback that delivers the request headers, but the trap needs the request body, which arrives in a later callback; a local answer from that callback is rejected, and pausing the request in the header callback stops it altogether. This variant therefore lets the request continue to the origin server, which has no `/api/login` and answers with status 404, examines the body when it arrives, and replaces the origin's reply with the trap page in the response phase. The client receives the same response as from the other variants (Section 5.7), at the cost of one additional round trip to the origin server.

5.5 Apache httpd with mod_lua

In this variant, RPDM consists of three Lua scripts, run by `mod_lua`, that share a common Lua library. The access-checker hook performs steps 1, 3 and 4 and also prepares the decoy header and cookie, in a set of response headers that Apache keeps when it forwards the request. A handler mapped to `/api/login` runs the trap, and an output filter inserts the honeytokens into the body after collecting it piece by piece. The `Content-Length` header is removed by the directive `Header always unset Content-Length` in the Apache configuration, so that the length of an injected body cannot contradict it. With the setting `LuaScope thread`, each script has its own Lua state per thread, which is reused for every request that thread serves (Section 2.6); the shared library therefore compiles the configuration once per state. Chapter 7 returns to this choice when it discusses the stability of Apache.

5.6 Levelling the Variants

R5 requires that the variants behave identically. A fair comparison also requires that no variant pays for work that the others do not do. Both were achieved by levelling the variants in three respects.

Behaviour. The first implementations, although written against the same configuration, did not behave identically. They used four different procedures to find and remove the keywords of the HTML comments. Apache found the keyword but never removed it, because its removal step treated the keyword as a Lua pattern, in which the characters `-` and `.` have a special meaning. Both Envoy variants took the client address from the `X-Forwarded-For` header, which the load generator does not send, and therefore recorded every client as unknown. The Envoy Lua variant also wrote an extra log line on every request after the first detection. All four variants were rewritten to follow the procedure of Section 4.5, and both Envoy configurations now take the client address from the connection.

Product defaults. Where the products differ in a default that affects cost, the default was aligned. Unlike Envoy and Apache, nginx does not reuse its connections to the origin server unless configured to (Section 2.4): over 100 requests sent on one client connection, OpenResty closed a connection to the origin after every request, while Envoy and Apache closed none. Connection reuse was therefore enabled in OpenResty. OpenResty and Apache wrote an access-log line for every request, which Envoy does not, so these logs were removed; the origin server's log still records every forwarded request. Apache by default checks every script for changes each time it runs it, while the other variants load their code once, so Apache was configured to load its scripts once.

Configurations without RPDM. Each variant's configuration without RPDM, which Chapter 6 uses as the baseline, was derived from its configuration with RPDM by deleting the RPDM parts and nothing else, and it runs in the same container. Two differences are kept on purpose, because they are part of what RPDM costs: injected responses are larger, and Apache sends them without a `Content-Length` header.

Any change made during levelling was followed by a new measurement of both configurations of the variant concerned, so that no result compares a levelled variant with one that was not.

5.7 Verifying Equivalence

An automated parity check sends the same 13 requests to every variant: the four `GET` requests of the benchmark (Chapter 6) and nine further cases, among them encoded keywords, the order of the remaining parameters after a keyword is removed, duplicated form fields, a path below the decoy path, and a signature hit and a clean login at the trap (Appendix B). For each request, it records what the client receives (status, decoy header, cookies and a hash of the body), the alert lines the variant writes, and the request that reaches the origin server, as recorded in the origin's log. It compares these with OpenResty.

Table 5.2 lists the differences that remain and are excluded from the comparison. None of them changes the response body a client receives or the alerts a variant writes. Run on the code that produced the results of Chapter 7, the check found no other difference from OpenResty: for the 13 requests, each variant wrote the same 11 alert lines and one trap line and, apart from the trap requests of the WebAssembly variant, forwarded the same 11 requests to the origin server.

Difference	Variant	Reason
Framing of injected responses	Envoy, Lua against the other three	Only the Envoy Lua variant knows the final length before it sends the headers (Section 5.3)
The origin receives `path?` instead of `path` when every parameter was removed	Apache	`mod_lua` can set the query only to a string, and Apache then appends `?`; the origin serves the same resource
At most one cookie honeytoken per path	Apache	`mod_lua` cannot add a second `Set-Cookie` header in this hook; the configuration plants one
The trap request reaches the origin server	Envoy, WebAssembly	A constraint of the module interface (Section 5.4); the client's response is identical

Table 5.2: Differences between the variants that remain after levelling.

6 Evaluation Methodology

How the costs were measured, and why the measurements can be trusted.

6.1 Goals and Metrics

A table mapping each metric to its question. RQ1: latency added by RPDM (median and p95) and latency above the origin server. RQ2: CPU per request, capacity. RQ3: share of intact responses, alert recall, false positives. RQ4: in-proxy timer cost per honeytoken type.

6.2 Testbed

The VM, the cpuset layout, guest tuning, image versions.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4

6.3 Workload

k6 iterations of nine requests: a priming `GET`, four `GET` probes and four SQLi `POST`s. Williams-balanced order [45]; arrival rates of 1, 10 and 100 iterations/s.

Material: `benchmarks/test.js`

6.4 Tiers and Pairing

Four tiers per replicate, in an order balanced across replicates: *origin* (load generator straight to the origin server, host proxy idle), *bare* (host proxy without RPDM), *silent* (RPDM as deployed) and *instrumented* (RPDM plus timing log lines). Differences: bare − origin is the cost of the host proxy, silent − bare the cost of RPDM, silent − origin that of the deployed stack, instrumented − silent the cost of measuring. The

origin tier is run only for the main workload, not for the SQL-injection experiment (the origin server has no trap), and records no host-proxy CPU.

Material: `benchmarks/run_paired_benchmark.py`, `benchmarks/edge_specs.py`

6.5 Measurement Planes

In-proxy timers (µs), end-to-end latency from k6 (ms), CPU per request from cgroup `cpu.stat`.

6.6 Run Validity

The `assess_run()` gates, the origin-CPU negative control, and detection of host freezes.

Material: `benchmarks/wadm_timings.py`, `docs/THESIS_NOTES.md` (2026-10-07)

7 Results

One section per research question, each ending with its answer.

7.1 RQ1: Latency Added

7.1.1 Median Latency per GET across Rates

At 1 and 10 iterations/s, +110 to +233 µs per `GET` (12–27% of bare), resolved in every `GET` cell. At 100 iterations/s, +111 to +147 µs (18–22%) in three variants; Apache +295 µs (+48%).

Material: `results/paired_headline.md`, `plots/paired_latency_added.png`

7.1.2 Latency above the Origin Server

The host proxy without RPDM adds +184 to +396 µs per `GET` over direct access to the origin server. RPDM accounts for 23–42% of the latency the deployed stack adds above the origin at 1 and 10 iterations/s; at 100 iterations/s, 28–37% in three variants and 51% on Apache.

Material: `results/origin_floor.md`

7.1.3 Tail Latency

The p95 difference exceeds the median difference in every cell except Envoy+Lua at 10 iterations/s. Apache at 100 iterations/s: median +295 µs, p95 +83,946 µs.

7.1.4 Request Position and Reconnects

The first request after the think-time sleep pays a wake-up cost. Apache closes the connection after the trap's 500.

Material: `docs/THESIS_NOTES.md` (2026-09-21, §5b), `plots/paired_latency_shift.png`

7.1.5 Answer to RQ1

One paragraph: the number per variant, how it changes with load, and how certain it is.

7.2 RQ2: CPU and Capacity

7.2.1 CPU per Request

Marginal CPU added per request: OpenResty +31, Envoy+Lua +45, WASM +108 µs. For Apache the fit is rejected as unphysical; its CPU added per request is +295, +129 and +495 µs at 1, 10 and 100 iterations/s.

7.2.2 Cost Attribution

The in-proxy timers cover 5–22% of that CPU**[TODO: re-check against the 2026-10-15 dataset]**; the rest is VM entry and exit, body filters and log writes.

Material: `plots/cost_attribution.png`

7.2.3 The Cost of Measuring

At 10 and 100 iterations/s, the timing log lines add 26–41 µs of CPU per request on OpenResty and Envoy+Lua, against 31–55 µs for RPDM itself.

7.2.4 Per-Mechanism Cost and Page Size

The `GET` path against the trap; injection cost grows linearly with body size up to 768 KiB.

Material: `results/mechanisms.md`, `plots/mechanism_cpu.png`, `plots/size_sweep.png`

7.2.5 Answer to RQ2

CPU and capacity per variant, and where the cost arises.

7.3 RQ3: Functional Correctness

7.3.1 Response Integrity under Load

Share of intact responses per probe and rate. The origin tier is 100% intact at every rate; Apache is the only variant that falls below 100%.

Material: `results/paired_functional.md`

7.3.2 Detection Recall and False Positives

`WADM ALERT` and `WADM TRAP` lines counted against expected; zero on bare.

7.3.3 Answer to RQ3

What breaks, in which variant, and whether detection is complete.

7.4 RQ4: Influence of the Extension Runtime

7.4.1 Per-Type Operation Cost

In-proxy timer distributions per honeytoken type and variant.

Material: `plots/paired_internal_timers.png`

7.4.2 The SQLi Trap: Scan Depth versus Decoding

In the encoding-by-outcome 2×2, decoding costs 0.5–2.5 µs in the Lua-based variants and nothing on WASM; scan depth is visible only on Apache's interpreted PUC Lua.

Material: `results/sqli.md`, `docs/THESIS_NOTES.md` (2026-09-21)

7.4.3 Stability of Apache mod_lua

Segfaults driven by cumulative requests in the body filter, and the cost of fixing them with `LuaScope conn`.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4b–4c, `plots/apache_remediation.png`

7.4.4 Answer to RQ4

How much of RQ1–RQ3 is explained by the runtime rather than by the deception logic.

8 Discussion

What the results mean, and how far they can be trusted.

8.1 Synthesis across Research Questions

One table of variants against the four answers.

8.2 Choosing a Variant for Deception

A trade-off summary per variant: latency, CPU, capacity, robustness.

8.3 Where the Cost Comes From

Runtime entry and exit, body buffering and log writes, not the detection logic.

8.4 Deployment Implications

Compressed bodies, range requests, preserved ETags, body-size limits.

8.5 Threats to Validity

A single VM host, synthetic traffic, moving image tags, timer resolution.

8.6 Limitations

POST-body inspection exists in only two variants; no real attacker traffic; no TLS or HTTP/2.

9 Conclusion and Future Work

9.1 Summary

RQ1–RQ4 answered in one or two sentences each, then the overall contribution.

9.2 Future Work

POST-body inspection in all four variants, injecting into compressed bodies, a field study with real attackers, TLS and HTTP/2.

A Configuration Reference

Every `config.json` field per honeytoken type, and the SQL-injection trap's fields.

Material: `CONTEXT.md`

B Parity-Check Request Set

The 13 requests and the documented exclusions.

C Reproducing the Benchmarks

Commands, machine configuration, run durations.

Material: `docs/BENCHMARK_METHODOLOGY.md` §3–4

D Additional Result Tables

Per-rate tables and figures.

Material: `benchmarks/results/plots/by_rate/`

Sources Used for This Proposal

[1] Verizon Business, "2026 data breach investigations report." https://web.archive.org/web/20261014053535/https://www.verizon.com/business/resources/reports/dbir/, 2026. Accessed: 18.10.2026.

[2] L. Zhang and V. L. L. Thing, "Three decades of deception techniques in active cyber defense – retrospect and outlook," *Computers & Security*, vol. 106, p. 102288, 2021.

[3] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000.

[4] D. E. Denning, "Framework and principles for active cyber defense," *Computers & Security*, vol. 40, pp. 108–113, 2014.

[5] OWASP Foundation, "About the OWASP foundation." https://web.archive.org/web/20261013085208/https://owasp.org/about/. Accessed: 18.10.2026.

[6] OWASP Foundation, *Web Security Testing Guide, Version 4.2*, 2020. Tests WSTG-INFO-03, WSTG-INFO-05 and WSTG-INPV-05. Release of 03.12.2020: https://github.com/OWASP/wstg/releases/tag/v4.2. Accessed: 12.10.2026.

[7] X. Han, N. Kheir, and D. Balzarotti, "Evaluation of deception-based web attacks detection," in *Proceedings of the 2017 Workshop on Moving Target Defense (MTD)*, pp. 65–73, 2017.

[8] M. Kahlhofer and S. Rass, "Application layer cyber deception without developer interaction," in *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 416–429, 2024.

[9] nginx, "Module ngx_http_proxy_module." https://web.archive.org/web/20261003190217/https://nginx.org/en/docs/http/ngx_http_proxy_module.html. Accessed: 18.10.2026.

[10] The Apache Software Foundation, "Apache module mod_proxy, apache HTTP server version 2.4 documentation." https://web.archive.org/web/20261005043655/https://httpd.apache.org/docs/2.4/mod/mod_proxy.html. Accessed: 18.10.2026.

[11] OpenResty, "lua-nginx-module: Embed the power of Lua into NGINX HTTP servers." https://web.archive.org/web/20260905133050/https://github.com/openresty/lua-nginx-module. Accessed: 13.10.2026.

[12] The Apache Software Foundation, "Apache module `mod_lua`, apache HTTP server version 2.4 documentation." `https://web.archive.org/web/20260915203410/https://httpd.apache.org/docs/2.4/mod/mod_lua.html`. Accessed: 13.10.2026.

[13] The Envoy Project Authors, "What is Envoy, envoy 1.30 documentation." `https://web.archive.org/web/20260415194608/https://www.envoyproxy.io/docs/envoy/v1.30.0/intro/what_is_envoy`. Accessed: 18.10.2026.

[14] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, "Dissecting overheads of service mesh sidecars," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pp. 142–157, 2023.

[15] OpenResty Inc., "OpenResty." `https://web.archive.org/web/20261013082229/https://openresty.org/en/`. Accessed: 18.10.2026.

[16] Lua.org, PUC-Rio, "About Lua." `https://web.archive.org/web/20261008163524/https://www.lua.org/about.html`. Accessed: 18.10.2026.

[17] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics." RFC 9110, Internet Engineering Task Force, June 2022.

[18] T. Berners-Lee, R. T. Fielding, and L. Masinter, "Uniform resource identifier (URI): Generic syntax." RFC 3986, Internet Engineering Task Force, Jan. 2005.

[19] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP/1.1." RFC 9112, Internet Engineering Task Force, June 2022.

[20] WHATWG, "HTML living standard, section 4.10.22: Form submission." `https://web.archive.org/web/20261013090759/https://html.spec.whatwg.org/multipage/form-control-infrastructure.html`. Accessed: 18.10.2026.

[21] A. Barth, "HTTP state management mechanism." RFC 6265, Internet Engineering Task Force, Apr. 2011.

[22] WHATWG, "HTML living standard, section 13.1.6: Comments." `https://web.archive.org/web/20261013081125/https://html.spec.whatwg.org/multipage/syntax.html`. Accessed: 18.10.2026.

[23] WHATWG, "HTML living standard, section 4.10.5.1.1: Hidden state." `https://web.archive.org/web/20261013122918/https://html.spec.whatwg.org/multipage/input.html`. Accessed: 18.10.2026.

[24] X. Han, N. Kheir, and D. Balzarotti, "Deception techniques in computer security: A research perspective," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–36, 2018.

[25] L. Spitzner, "Honeypots: Catching the insider threat," in *Proceedings of the 19th Annual Computer Security Applications Conference (ACSAC)*, pp. 170–179, 2003.

[26] nginx, "nginx." https://web.archive.org/web/20261015150421/https://nginx.org/en/. Accessed: 18.10.2026.

[27] The Envoy Project Authors, "HTTP filters, envoy 1.30 documentation." https://web.archive.org/web/20250615000955/https://www.envoyproxy.io/docs/envoy/v1.30.0/intro/arch_overview/http/http_filters. Accessed: 13.10.2026.

[28] The Envoy Project Authors, "Lua filter, envoy 1.30 documentation." https://web.archive.org/web/20250810044910/https://www.envoyproxy.io/docs/envoy/v1.30.0/configuration/http/http_filters/lua_filter. Accessed: 13.10.2026.

[29] The Envoy Project Authors, "Wasm filter, envoy 1.30 documentation." https://web.archive.org/web/20250713074605/https://www.envoyproxy.io/docs/envoy/v1.30.0/configuration/http/http_filters/wasm_filter. Accessed: 13.10.2026.

[30] Proxy-Wasm Project, "WebAssembly for proxies (ABI specification)." https://web.archive.org/web/20260830061200/https://github.com/proxy-wasm/spec. Accessed: 13.10.2026.

[31] The Envoy Project Authors, "Wasm, envoy documentation." https://web.archive.org/web/20260513065059/https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/advanced/wasm. Accessed: 13.10.2026.

[32] R. Ierusalimschy, L. H. de Figueiredo, and W. Celes, "The implementation of Lua 5.0," *Journal of Universal Computer Science*, vol. 11, no. 7, pp. 1159–1176, 2005.

[33] M. Pall, "LuaJIT." https://web.archive.org/web/20261012063453/https://luajit.org/luajit.html. Accessed: 13.10.2026.

[34] M. Pall, "Running LuaJIT." https://web.archive.org/web/20261011192338/https://luajit.org/running.html. Option `-O`, parameter `hotloop`. Accessed: 13.10.2026.

[35] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 185–200, 2017.

[36] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pp. 1–12, 2015.

[37] Docker Inc., "What is Docker?." https://web.archive.org/web/20261012110729/https://docs.docker.com/get-started/docker-overview/. Accessed: 18.10.2026.

[38] The Linux Kernel Documentation, "Control group v2." `https://web.archive.org/web/20261007110448/https://docs.kernel.org/admin-guide/cgroup-v2.html`. Accessed: 18.10.2026.

[39] M. Sahin, C. Hebert, and A. S. de Oliveira, "Lessons learned from SunDEW: A self defense environment for web applications," in *Proceedings of the Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, 2020.

[40] D. Fraunholz, D. Reti, S. D. Anton, and H. D. Schotten, "Cloxy: A context-aware deception-as-a-service reverse proxy for web services," in *Proceedings of the 5th ACM Workshop on Moving Target Defense (MTD)*, pp. 40–47, 2018.

[41] M. Kahlhofer, M. Golinelli, and S. Rass, "Koney: A cyber deception orchestration framework for Kubernetes," in *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 690–702, 2025.

[42] D. Reti, T. Angeli, and H. D. Schotten, "Honey infiltrator: Injecting honeytoken using netfilter," in *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 465–469, 2023.

[43] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 310–325, 2019.

[44] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 265–276, 2009.

[45] E. J. Williams, "Experimental designs balanced for the estimation of residual effects of treatments," *Australian Journal of Scientific Research, Series A*, vol. 2, no. 2, pp. 149–168, 1949.